Architectures in Word2Vec

1-Skip-Gram

An important aspect of Natural Language Processing (NLP) is capturing the relationships between words. Imagine words existing in a vast space, where similar words are positioned closer together. This is where word embeddings come in. They represent words as numerical vectors, allowing computers to understand the semantic connections between them. One technique for creating word embeddings is called skip-gram.

Understanding Word Embeddings

Imagine trying to teach a friend a new language. But instead of memorizing definitions, you show them how words connect and relate to each other. Word embeddings do something similar for computers. Instead of just storing dictionary definitions, they capture the essence of a word by its relationship to other words.

Think of words as points scattered around a giant map. Words with similar meanings end up closer together on this map. For instance, “king” and “queen” would be neighbors, while “king” and “banana” might be much farther apart. This way, computers can analyze the closeness of words in this embedding space to understand their semantic relationships. Word embeddings are like a secret code that unlocks the meaning of words for machines, allowing them to perform amazing tasks like machine translation and sentiment analysis.

Unveiling Skip-Gram

Skip-gram is a technique for creating word embeddings that focuses on predicting surrounding words based on a specific word, also called the “target word.” Imagine you’re reading a sentence and can guess the words that come before and after a particular word. Skip-gram essentially does the same thing, but on a massive scale with computer programs. By analyzing how well it predicts surrounding words, skip-gram creates a numerical representation (embedding) that captures the meaning and context of the target word.

For example, consider the sentence “The **king** wore a golden **crown**.” In skip-gram, “king” would be the target word. The model would then try to predict words like “wore” and “crown” that appear around “king” in the sentence. This prediction process helps the model understand the meaning and relationships of “king” with other words. We’ll explore a clearer example with an actual sentence in the next section.

Deepening Our Dive

While we haven’t delved into the technical details, it’s important to note that skip-gram leverages a simple neural network architecture to perform its predictions. This neural network is trained on a massive dataset of text, where it learns to predict surrounding words based on the target word. The

better the model predicts these surrounding words, the more accurate the resulting word embedding becomes.

Skip-gram offers some advantages compared to other word embedding methods. For instance, it's known for its efficiency in handling large amounts of text data. Additionally, skip-gram can capture both syntactic (grammatical) and semantic (meaning-based) relationships between words. This makes the resulting word embeddings more versatile for various NLP tasks.

The Power of Skip-gram

Skip-gram isn't just a cool concept; it has real-world applications in various NLP tasks. Here are a few examples:

Machine translation: By understanding the relationships between words, skip-gram embeddings can help translate sentences from one language to another while preserving meaning and context.

Sentiment analysis: Skip-gram embeddings can be used to analyze the sentiment of text data. By understanding the emotional tone of surrounding words, the model can determine if a sentence expresses positive, negative, or neutral sentiment.

Question answering: Skip-gram embeddings can play a role in question answering systems. By analyzing the relationships between words in a question and a passage, the system can identify the most relevant answer within the text.

How Skip-Gram works?

(1) Model Structure:

The Skip-Gram model is basically a shallow neural network with an input layer, embedding layer and output layer. The model's aim is to produce an output probability distribution vector given a target word input. This probability distribution vector (which sums to 1) reflects the probability for each word to appear in the target word's context window. As one might expect, we are interested in locating the probability to be high for words who share the same context and low for words who don't.

The intriguing part about this is that we just require the model's weights once it has been trained.

You may wonder how the model accounts for the (target word, context word) pairings that were previously described. The explanation is that the target words function as the input (X_train), while the context words function as the output (y_train).

(2) The Loss Function:

To obtain useful vector embeddings, we have to optimize the weights in our model, which are initially set to be entirely random. The optimization process is carried out in order to minimize the loss function:

$$J = -\sum_{t=1}^{T} \sum_{-m \leq j \leq m} log(P(w_{t+j}|w_t))$$

Our Loss Function. T = text length, m = window size, P(wt+j|wt) = the probability to get context word given target word.

How to read this equation? It is merely a nested loop, which requires us to iterate over all (target word, context word) pairs and sum up the probabilities. The minus sign is just for convenience — in ML we prefer minimizing the loss function's value instead of maximizing it.

(3) How are these probabilities calculated?

In order to calculate the probability distribution, we use the Softmax function which takes into account the dot product of the target embedding vector and embedding vectors of each and every word in our vocabulary:

$$u^T v = u \cdot v = \sum_{i=1}^{n} u_i v_i$$

Dot product of two vectors

Press enter or click to view image in full size

$$p(context\ word/target\ word) = \frac{exp(u^T_{target} v_{context})}{\Sigma_{w=1}^{Words}(exp(u^T_{target} v_w))}$$

The softmax function which provide probability distribution

Some notes and clarifications before we move on to the implementation:

1. **Regarding the model purpose** — As opposed to a traditional neural network that is used for classification, over here our objective is to train the network in order to extract the weights. In summary, there is no need to use it in the traditional manner of input -> model -> output.

2. **Regarding the model's weights** — As opposed to a typical neural network, where the weights are used to handle any input, the skip-gram model assigns unique weights to each and every word. This

means that the model's weights cannot be generalized to handle new words or inputs in the same way. Instead, the weights serve as specific embeddings for each word.

3. **Using one hot encoding is not required** — According to various blog entries I've read, every word is converted into a one-hot encoding vector which we pass to the model (like 'rick' = [0,0,0,…1,…0]). Despite being the standard, using a sparse vector makes little sense. Instead, we can just pass the word's index which is less memory consuming.

---

2 -Continuous Bag of Words (CBOW)

Semantics is important because in NLP it is the relationships between the words that are being studied. One of the simplest yet highly effective procedure is Continuous Bag of Words (CBOW) which maps words to highly meaningful vectors called word vectors. CBOW is used in the Word2Vec framework and predicts a word based on the words that are adjacent to it which captures the semantic as well as syntactic meaning of language. In this article, the reader will learn about the operation of the CBOW model, as well as the methods of its use.

What is Continuous Bag of Words Model?

The Continuous Bag of Words (CBOW) is also a model that is used when determining word embedding using a neural network and is part of Word2Vec models by Tomas Mikolov. CBOW tries to predict a target word depending on the context words observing it in a given sentence. This way it is able to capture the semantic relations hence close words are represented closely in a high dimensional space.

For example, in the sentence **"The cat sat on the mat"**, if the context window size is 2, the context words for **"sat"** are **["The", "cat", "on", "the"]**, and the model's task is to predict the word **"sat"**.

CBOW operates by aggregating the context words (e.g., averaging their embeddings) and using this aggregate representation to predict the target word. The model's architecture involves an input layer for the context words, a hidden layer for embedding generation, and an output layer to predict the target word using a probability distribution.

It is a fast and efficient model suitable for handling frequent words, making it ideal for tasks requiring semantic understanding, such as text classification, recommendation systems, and sentiment analysis.

How Continuous Bag of Words Works

CBOW is one of the simplest, yet efficient techniques as per context for word embedding where the whole vocabulary of words are mapped to vectors. This section also describes the operation of the CBOW system as a means of comprehending the method at its most basic level, discussing the main ideas that underpin the CBOW method, as well as offering a comprehensive guide to the architectural layout of the CBOW hit calculation system.

Understanding Context and Target Words

CBOW relies on two key concepts: context words and the target word.

- **Context Words**: These are the words surrounding a target word within a defined window size. For example, in the sentence:
  "The quick brown fox jumps over the lazy dog",
  if the target word is **"fox"** and the context window size is 2, the context words are **["quick", "brown", "jumps", "over"]**.

- **Target Word**: This is the word that CBOW aims to predict, given the context words. In the above example, the target word is **"fox"**.

By analyzing the relationship between context and target words across large corpora, CBOW generates embeddings that capture semantic relationships between words.

Step-by-Step Process of CBOW

Here's a breakdown of how CBOW works, step-by-step:

Step1: Data Preparation

- Choose a corpus of text (e.g., sentences or paragraphs).

- Tokenize the text into words and build a vocabulary.

- Define a context window size nnn (e.g., 2 words on each side).

Step2: Generate Context-Target Pairs

- For each word in the corpus, extract its surrounding context words based on the window size.

- Example: For the sentence **"I love machine learning"** and n=2n = 2n=2, the pairs are:**Target WordContext Words**love["I", "machine"]machine["love", "learning"]

Step3: One-Hot Encoding

Convert the context words and target word into one-hot vectors based on the vocabulary size. For a vocabulary of size 5, the one-hot representation of the word "love" might look like **[0, 1, 0, 0, 0]**.

Step4: Embedding Layer

Pass the one-hot encoded context words through an embedding layer. This layer maps each word to a dense vector representation, typically of a lower dimension than the vocabulary size.

Step5: Context Aggregation

Aggregate the embeddings of all context words (e.g., by averaging or summing them) to form a single context vector.

Step6: Prediction

- Feed the aggregated context vector into a fully connected neural network with a softmax output layer.
- The model predicts the most probable word as the target based on the probability distribution over the vocabulary.

Step7: Loss Calculation and Optimization

- Compute the error between the predicted and actual target word using a cross-entropy loss function.
- Backpropagate the error to adjust the weights in the embedding and prediction layers.

Step8: Repeat for All Pairs

Repeat the process for all context-target pairs in the corpus until the model converges.

CBOW Architecture Explained in Detail

The Continuous Bag of Words (CBOW) model's architecture is designed to predict a target word based on its surrounding context words. It is a shallow neural network with a straightforward yet effective structure. The CBOW architecture consists of the following components:

Input Layer

- **Input Representation**:
  The input to the model is the context words represented as **one-hot encoded vectors**.
  - If the vocabulary size is V, each word is represented as a one-hot vector of size V with a single 1 at the index corresponding to the word, and 0s elsewhere.
  - For example, if the vocabulary is ["cat", "dog", "fox", "tree", "bird"] and the word "fox" is the third word, its one-hot vector is [0,0,1,0,0][0, 0, 1, 0, 0][0,0,1,0,0].
- **Context Window**:
  The context window size n determines the number of context words used. If n=2, two words on each side of the target word are used.
  - For a sentence: **"The quick brown fox jumps over the lazy dog"** and target word **"fox"**, the context words with n=2 are **["quick", "brown", "jumps", "over"]**.

Embedding Layer

- **Purpose**:
  This layer converts one-hot vectors which exist in a high dimension into maximally dense and low dimensions vectors. In contrast to the fact that in word embedding words are represented as vectors with mostly zero values, in the embedding layer, each word is encoded by the continuous vector of the required dimensions that reflects specific characteristics of the word meaning.

- **Word Embedding Matrix**:
  The embedding layer maintains a word embedding matrix W of size V×d, where V is the vocabulary size and d is the embedding dimension.

  - Each row of W represents the embedding of a word.

  - For a one-hot vector xxx, the embedding is computed as W^T X x.

- **Context Word Embeddings**:
  Each context word is transformed into its corresponding dense vector using the embedding matrix. If the window size n=2, and we have 4 context words, the embeddings for these words are extracted.

Hidden Layer: Context Aggregation

- **Purpose**:
  The embeddings of the context words are combined to form a single context vector.

- **Aggregation Methods**:

  - **Averaging**: The embeddings of all context words are averaged to compute the context vector.

$$h = \frac{1}{2n} \sum_{i=1}^{2n} e_i$$

- **Summation**: Instead of averaging, the embeddings are summed.

$$h = \sum_{i=1}^{2n} e_i$$

- **Resulting Context Vector**: The result is a single dense vector hhh, which represents the aggregated context of the surrounding words.

Output Layer

- **Purpose**: The output layer predicts the target word using the context vector hhh.

- **Fully Connected Layer**: The context vector hhh is passed through a fully connected layer, which outputs a raw score for each word in the vocabulary. These scores are called logits.

- **Softmax Function**: The logits are passed through a softmax function to compute a probability distribution over the vocabulary:

$$\bar{h} = \frac{[0.1, 0.2, 0.3] + [0.4, 0.5, 0.6] + [0.2, 0.3, 0.4]}{3} = [0.233, 0.333, 0.433]$$

- **Predicted Target Word**: The first cause is that at the softmax output, the algorithm defines the target word as the word with the highest probability.

Loss Function

- The cross-entropy loss is used to compare the predicted probability distribution with the actual target word (ground truth).
- The loss is minimized using optimization techniques like Stochastic Gradient Descent (SGD) or its variants.

Example of CBOW in Action

Input:
Sentence: **"I love machine learning"**, target word: **"machine"**, context words: **["I", "love", "learning"]**.

One-Hot Encoding:
Vocabulary: **["I", "love", "machine", "learning", "AI"]**

- One-hot vectors:
  - "I": [1,0,0,0,0][1, 0, 0, 0, 0][1,0,0,0,0]
  - "love": [0,1,0,0,0][0, 1, 0, 0, 0][0,1,0,0,0]
  - "learning": [0,0,0,1,0][0, 0, 0, 1, 0][0,0,0,1,0]

Embedding Layer:

- Embedding dimension: d=3.
- Embedding matrix W:

$$W = \begin{bmatrix} 0.1 & 0.2 & 0.3 \\ 0.4 & 0.5 & 0.6 \\ 0.7 & 0.8 & 0.9 \\ 0.2 & 0.3 & 0.4 \\ 0.5 & 0.6 & 0.7 \end{bmatrix}$$

Embeddings:

- "I": [0.1,0.2,0.3]
- "love": [0.4,0.5,0.6]

- “learning”: [0.2,0.3,0.4]

Aggregation:

- Average the embeddings:

$$\breve{h} = \frac{[0.1, 0.2, 0.3] + [0.4, 0.5, 0.6] + [0.2, 0.3, 0.4]}{3} = [0.233, 0.333, 0.433]$$

Output Layer:

- Compute logits, apply softmax, and predict the target word.

Diagram of CBOW Architecture

```
Input Layer: ["I", "love", "learning"]
    --> One-hot encoding
    --> Embedding Layer
        --> Dense embeddings
        --> Aggregated context vector
        --> Fully connected layer + Softmax
Output: Predicted word "machine"
```

Advantages of Continuous Bag of Words

We will now explore advantages of continuous bag of words:

- **Efficient Learning of Word Representations**: CBOW efficiently learns dense vector representations for words by using context words. This results in lower-dimensional vectors compared to traditional one-hot encoding, which can be computationally expensive.

- **Captures Semantic Relationships**: CBOW captures semantic relationships between words based on their context in a large corpus. This allows the model to learn word similarities, synonyms, and other contextual nuances, which are useful in tasks like information retrieval and sentiment analysis.

- **Scalability**: The CBOW model is highly scalable and can process large datasets efficiently, making it well-suited for applications with vast amounts of text data, such as search engines and social media platforms.

- **Contextual Flexibility**: CBOW can handle varying amounts of context (i.e., the number of surrounding words considered), offering flexibility in how much context is required for learning the word representations.

- **Improved Performance in NLP Tasks**: CBOW's word embeddings enhance the performance of downstream NLP tasks, such as text classification, named entity recognition, and machine translation, by providing high-quality feature representations.

Limitations of Continuous Bag of Words

Let us now discuss the limitations of CBOW:

- **Sensitivity to Context Window Size**: The performance of CBOW is highly dependent on the context window size. A small window may result in capturing only local relationships, while a large window may blur the distinctiveness of words. Finding the optimal context size can be challenging and task-dependent.

- **Lack of Word Order Sensitivity**: CBOW disregards the order of words within the context, meaning it doesn't capture the sequential nature of language. This limitation can be problematic for tasks that require a deep understanding of word order, like **syntactic parsing** and **language modeling**.

- **Difficulty with Rare Words**: CBOW struggles to generate meaningful embeddings for rare or out-of-vocabulary (OOV) words. The model relies on context, but sparse data for infrequent words can lead to poor vector representations.

- **Limited to Shallow Contextual Understanding**: While CBOW captures word meanings based on surrounding words, it has limited capabilities in understanding more complex linguistic phenomena, such as long-range dependencies, irony, or sarcasm, which may require more sophisticated models like transformers.

- **Inability to Handle Polysemy Well**: Words with multiple meanings (polysemy) can be problematic for CBOW. Since the model generates a single embedding for each word, it may not capture the different meanings a word can have in different contexts, unlike more advanced models like **BERT** or **ELMo**.

Conclusion

The Continuous Bag of Words (CBOW) model has proven to be an efficient and intuitive approach for generating word embeddings by leveraging surrounding context. Through its simple yet effective architecture, CBOW bridges the gap between raw text and meaningful vector representations, enabling a wide range of NLP applications. By understanding CBOW's working mechanism, its strengths, and limitations, we gain deeper insights into the evolution of NLP techniques. With its foundational role in embedding generation, CBOW continues to be a stepping stone for exploring advanced language models.

Optimization Technique

Negative Sampling

When working with natural language processing (NLP) tasks, especially word embeddings, efficiently optimizing models like Word2Vec becomes critical due to the vast vocabulary sizes involved. **Negative Sampling** is a powerful technique that simplifies and accelerates the training of such models by approximating the computationally expensive softmax function.

Why Do We Need Negative Sampling?

Imagine you are teaching a model the relationship between words in a sentence. For example:

- Sentence: "The dog chased the cat"

The model tries to learn:

1. "dog" is related to "chased".
2. "dog" is related to "cat".

But there's a problem:

- There are **millions of words** in the vocabulary.
- For every pair of words (e.g., "dog" and "chased"), the model has to compute how likely "dog" is related to **all other words** in the vocabulary. This is extremely slow.

Solution: Instead of looking at all words, Negative Sampling only looks at:

1. The **correct word pairs** (positive samples).
2. A few **random wrong word pairs** (negative samples).

This makes the training much faster!

How Does Negative Sampling Work?

Let's understand it with a step-by-step example.

Step 1: Vocabulary

Assume the vocabulary has only 5 words:
["dog", "cat", "chased", "apple", "banana"].

Step 2: Training Data

From the sentence "The dog chased the cat", the model creates pairs of related words:

- **Target word**: "dog"
- **Positive context words** (related): ["chased", "cat"].

Step 3: Negative Sampling

Instead of comparing "dog" to every word in the vocabulary, the model:

1. Picks the **positive pair**: "dog" and "chased".
2. Randomly selects a few **negative samples** (unrelated words):

- "dog" and "apple".
- "dog" and "banana".

Step 4: Training

The model now has two types of pairs:

Positive Pair:

- "dog" and "chased"
- The model should learn that they are related.

Negative Pairs:

- "dog" and "apple"
- "dog" and "banana"
- The model should learn that they are **not** related.

Step 5: Computation

For each pair:

Positive Pair: "dog" and "chased"

- The model tries to increase their similarity.

Negative Pairs: "dog" and "apple", "dog" and "banana"

- The model tries to decrease their similarity.

Key Point: Instead of comparing "dog" to **all 5 words** in the vocabulary, the model only works with 3 pairs:

- 1 positive pair.
- 2 negative pairs.

This drastically reduces the number of computations!

How Does This Save Time?

Without Negative Sampling:

- The model computes for **all 5 words** in the vocabulary for every training pair.

With Negative Sampling:

- The model only computes for **3 words** (1 positive + 2 negatives).

For larger vocabularies (like 1 million words), this means the model processes just a handful of words instead of all 1 million. Training becomes much faster.

When to Use Negative Sampling?

Use it when:

- You're working with large vocabularies (millions of words).
- You care about learning word embeddings quickly, not exact probabilities.

Advantages

1. **Faster Training**: Processes fewer words, saving time and resources.
2. **Simplicity**: Easy to implement.

Disadvantages

1. **Approximation**: Doesn't compute probabilities for all words, so it may miss rare words.
2. **Needs Fine-Tuning**: You must choose how many negative samples to use (e.g., 5 or 10).